VECTOR

Compiler Warning

Auxiliary Information

Version 1.5.0

Authors	Christian Kaiser
Status	Released

Document Information

History

Author	Date	Version	Remarks
Markus Schwarz	2019-10-02	1.2.0	Migrated from AN-ISC-8-1184_Compiler_Warnings
Christian Kaiser	2021-03-08	1.3.0	Added chapter 1.2 “Scope” CW_009, CW_010
Peter Wolf	2022-03-22	1.4.0	Clarified relevance of warnings in chapter 1.3
Christian Kaiser	2022-09-22	1.5.0	Added CW_011

Reference Documents

No.	Source	Title	Version
[MISRA-Compliance]	Vector	AuxiliaryInfo_MISRACompliance	1.01.00 or later
[TechRef_CSL]	Vector	Technical Reference MICROSAR ComStackLib	2.0.0 or later

Contents

1 Overview 5
1.1 Purpose 5
1.2 Scope 5
1.3 MICROSAR 5
1.4 Deviation Procedure 6
1.5 Default BSW Delivery Process 7

2 Accepted Deviations 8
2.1 Unused/Unreferenced Parameter/Argument 8
2.2 Unused/Unreferenced Define, Enum Value 9
2.3 Unused/Unreferenced Variable 9
2.4 Unused/Unreferenced Function 10
2.5 Condition Evaluates Always to True/False 10
2.6 Unreachable Code/Statement 11
2.7 Dead Assignment / Variable Set but Not Used 11
2.8 ASM Statements Used 12
2.9 Empty Source File 12
2.10 Conversion to type ‘abc’ from int may alter its value 13
2.11 Missing braces around initializer 13

3 Contact 14

Illustrations

Figure 1-1 MICROSAR BSW Component 5

Tables

Table 1-1 Priority of measures to prevent compiler warnings 6
Table 2-1 CW_001 8
Table 2-2 CW_002 9
Table 2-3 CW_003 9
Table 2-4 CW_004 10
Table 2-5 CW_005 10
Table 2-6 CW_006 11
Table 2-7 CW_007 11
Table 2-8 CW_008 12
Table 2-9 CW_009 12
Table 2-10 CW_010 13
Table 2-11 CW_011 13

1 Overview

1.1 Purpose

Warning free code is an important quality goal for embedded software. Nevertheless, compiler warnings can occur for highly configurable software. Typical compiler warnings are listed and justified within this document.

1.2 Scope

The scope of this documentation is embedded software implemented in the C programming language.

1.3 MICROSAR

MICROSAR BasicSoftware (BSW) is developed in a product line approach, independent from specific ECU projects or compilers. MICROSAR BSW supports the AUTOSAR MemoryAbstraction and CompilerAbstraction concepts and supports a huge range of different compiler vendors, versions and options.

MICROSAR BSW is delivered as static source code and code generators to support the ECU project specific adaption and optimization of the BSW behavior and feature set based on AUTOSAR configuration files and user defined selections.

Additionally, MICROSAR BSW supports compile time configuration, link time configuration and post-build configuration.

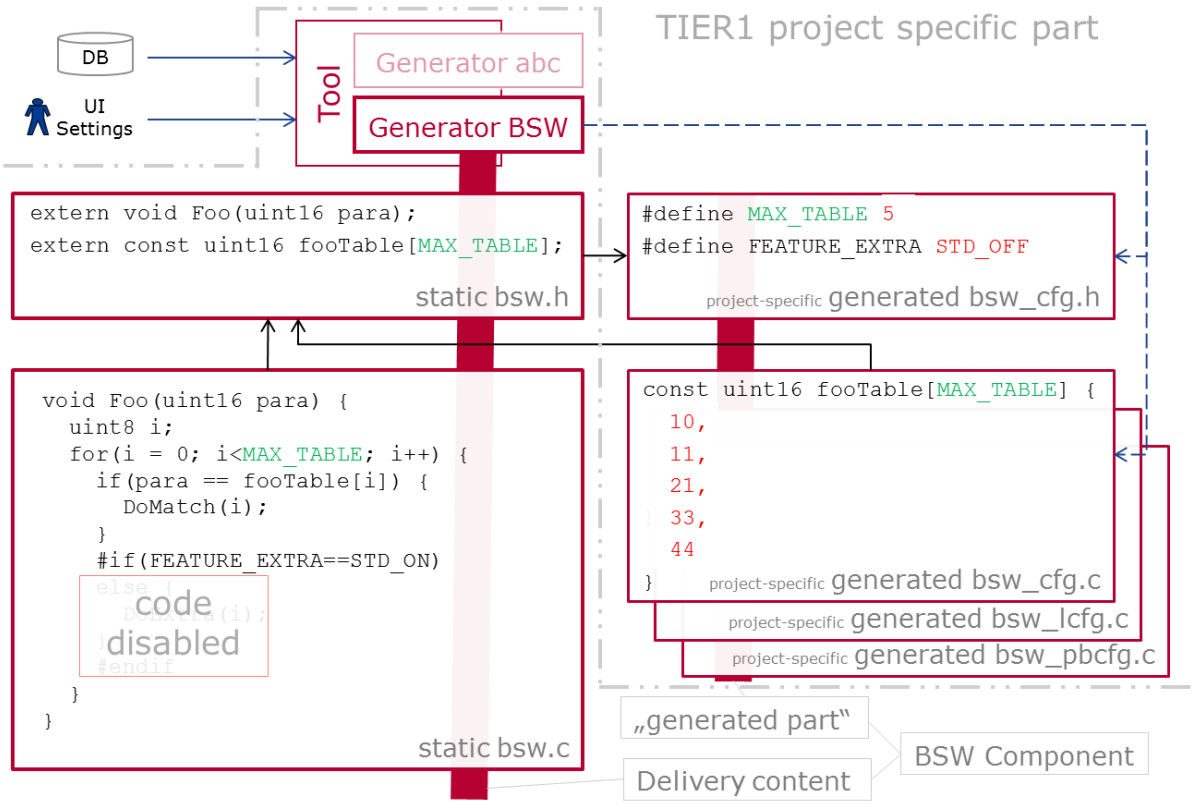


Figure 1-1 MICROSAR BSW Component

A general quality goal for MICROSAR BSW is “warning free code”. The coding style guide for MICROSAR BSW is based on MISRA-C:2004 and MISRA-C:2012. Checks for MISRA compliance are an essential part of our development process. (see [MISRA-Compliance])

Nevertheless, we accept deviations to the “warning free code” goal.

Due to the use of standard software modules for different hardware platforms and different compilers it is not possible to avoid compiler warnings completely. Some warnings are specialized for specific use cases and it is not feasible to activate all of them.

In addition, due to different and partly conflicting behaviors, warnings might occur only for some compilers. Options such as `-Werror` are therefore not supported.

Vector tries to keep its software free of warnings for a reasonable default set of active warnings, but in some cases it is not feasible. There are some common deviations that are defined within this document. And there are accepted MISRA deviations (see [MISRA-Compliance]) that might also be issued by some compilers depending on their warning level.

These deviations are caused by basic coding principles of our code. There is no ESCAN reporting for those deviations.

All other known and accepted deviations are reported within the delivery as ESCAN type “Warning in a released product”.

[TechRef_CSL] explains the relationship between user-selected code generation (optimization) options and compiler warnings.

1.4 Deviation Procedure

Priority	Rule	Rationale
1	MICROSAR BSW shall be compiler warning free. Thus, compiler warnings shall be prevented wherever possible.	Detection of compiler warning either during component development or more frequently when applying the customer specific compiler/options/BSW configuration during the delivery tests.
2	If a compiler warning is detected, the code shall be analyzed and, if reasonable, be changed/extended to become warning free.	We want to prevent to deliver defect code to our customers. We also need to take into account e.g. runtime efficiency, maintainability and standardized code for many use-cases.
3	If fixing the compiler warning is not a solution, an ESCAN of type “Warning in a released product” shall be created to document the accepted deviation also in the report of known issues of deliveries. Exception: The deviations in this document do not require an ESCAN, as they are reported to the customer with this document.	See list of accepted deviations in this document.

Table 1-1 Priority of measures to prevent compiler warnings

1.5 Default BSW Delivery Process

The configuration and compilation and linkage during the delivery test activities yield most of the compiler warnings, because the development tool chain and settings specified by the customer are used.
The process in short is:

- Customer specifies the compiler brand, version and options (via Questionnaire) and provides more information on the expected use-case
- Delivery engineer creates example configurations of the BSW and compiles and links the outcome
- Compiler warnings are analyzed and either documented as ESCAN of type “Warning in a released product” or reported to the development teams to trigger a fix.
- ESCANs of type “Issue in a released product” and “Warning in a released product” are reported within the delivery documentation as “known issues”

2 Accepted Deviations

Deviations in this chapter are typically accepted as “Warning in a released product” when

> it has been checked that no incorrect behavior at ECU runtime occurs

> no simple remedy exists



Note
If a compiler warning has passed the analyzing steps according Table 1 Priority of measures to prevent compiler warnings and the result is still “accepted deviation”, the compiler warning will not be fixed in the product.



Basic Knowledge
Generally, warning messages, i.e. their occurrences, locations, and contents, depend on components’ configurations. Building with a configuration *A* may produce different messages than building with a different configuration *B*.

Furthermore, some warnings may directly result from a single specific setting, especially user code or user-defined macros (e.g. as DUMMY_STATEMENT, referred to in CW_001 and CW_007).

2.1 Unused/Unreferenced Parameter/Argument

Deviation ID	CW_001
Example compiler warning strings	“Unreferenced formal parameters“, “parameter is never used“, “has no-used argument“
Reason	BSW provides APIs as described by standards. Not all parameters are necessary and used within the function in all possible usage scenarios. This warning is also covered by and MISRA-C:2012, rule 2.7, but whilst it is possible to justify such findings for a given analysis tool (e.g. QAC), this cannot be done in a portable way for compiler warnings.
Potential risk	The function contains unused code.
Prevention of risk	Code inspection has to check that the unused code is not critical concerning the expected behavior. Small increase of ROM footprint is accepted.
Note	Workarounds for suppressing such compiler warnings often conflict with MISRA-C:2012, rules 2.2 and 17.8 See [MISRA-Compliance] “MD_MSR_DummyStmt”.

Table 2-1 CW_001

2.2 Unused/Unreferenced Define, Enum Value

Deviation ID	CW_002
Example compiler warning strings	“Unused enumeration values“, “Unused define value“
Reason	Encapsulation of #defines, enum-values and similar items for multiple complex configuration data sets (via #ifdef) reduces readability and maintainability.
Potential risk	Defined but never used enums or defines may decrease readability and maintainability.
Prevention of risk	Execution of runtime tests with different configuration variants.
Note	-

Table 2-2 CW_002

2.3 Unused/Unreferenced Variable

Deviation ID	CW_003
Example compiler warning strings	“unreferenced local variable within abc.c“, “variable "abc" was declared but never referenced“, “Variables related to message supervision are set but never used“, “Unused constant” (in generated code)
Reason	Because of goal "minimize RAM footprint", unused variables shall be disabled via #ifdef based on configuration data sets. Nevertheless, warning can occur for rare cases where encapsulation for multiple complex configuration data sets (via #ifdef) reduces readability and maintainability.
Potential risk	The function contains unused code.
Prevention of risk	Code inspection has to check that the unused variable is not critical concerning the expected behavior. Small increase of RAM footprint is accepted.
Note	-

Table 2-3 CW_003

2.4 Unused/Unreferenced Function

Deviation ID	CW_004
Example compiler warning strings	“Declared but unused function abc“, “unused static function abc“
Reason	BSW provides APIs as described by standard. Because of goal "minimize ROM footprint", unused functions shall be disabled via #ifdef based on configuration data sets. Not all provided functions might be used in the concrete ECU project (configuration and usage dependent). Encapsulation for multiple complex configuration data sets (via #ifdef) reduces readability and maintainability and is thus not provided on a per-function base for all APIs.
Potential risk	The project contains unused code.
Prevention of risk	See “Reason”
Note	Partly addressed by MISRA-C:2012 Rule 2.1 See [MISRA-Compliance], “MD_MSR_Unreachable”.

Table 2-4 CW_004

2.5 Condition Evaluates Always to True/False

Deviation ID	CW_005
Example compiler warning strings	„conditional expression is constant“, „conditional expression […] always true/false / statement not reached“, „compare out of range / condition is always true/false“,
Reason	Typically caused by an if-statement applied on external configuration data. Configuration data is const for the given compilation context but might be changed at link-time or post-build time.
Potential risk	The function contains useless conditions with possibly dead code.
Prevention of risk	The code inspection is in charge to distinguish between useless conditions with possibly dead code and correct code as described in “reason”.
Note	Partly addressed by MISRA-C:2012 Rules 2.1, 2.2. See [MISRA-Compliance], “MD_MSR_Unreachable”, “MD_MSR_ConstantCondition”.

Table 2-5 CW_005

2.6 Unreachable Code/Statement

Deviation ID	CW_006
Example compiler warning strings	„conditional expression […] always true/false / statement not reached“, „statement will never be executed“, „this switch default label is unreachable“
Reason	Because of goal "minimize ROM footprint", unreachable code should be disabled via #ifdef based on configuration data sets. Nevertheless, warning might occur as secondary effect on e.g. "condition evaluates to true/false"
Potential risk	The function contains unused code.
Prevention of risk	Code inspection has to check that the unused code is not critical concerning the expected behavior. Small increase of ROM footprint is accepted.
Note	Partly addressed by MISRA-C:2012 Rules 2.1, 2.2. See [MISRA-Compliance], “MD_MSR_Unreachable”, “MD_MSR_ConstantCondition”.

Table 2-6 CW_006

2.7 Dead Assignment / Variable Set but Not Used

Deviation ID	CW_007
Example compiler warning strings	“Variable 'abc' was set but never used“, “local variable is initialized but not referenced”, “Dead assignment (<*details*>)“, “Useless assignment to variable …. Assigned value not used.”
Reason	Variable shall be always assigned a valid value. Overwriting the value before use can therefore happen based on the assumed execution path. This can especially occur for the DUMMY_STATEMENT macro we use or for the return value assignment in our API pattern.
Potential risk	The function contains unused code and variables.
Prevention of risk	Code inspection has to check that the unused code is not critical concerning the expected behavior. Small increase of ROM/RAM footprint is accepted.
Note	-

Table 2-7 CW_007

2.8 ASM Statements Used

Deviation ID	CW_008
Example compiler warning strings	“asm statements used“
Reason	MICROSAR is written in C but might use compiler specific ASM statements in rare cases. Usage of ASM is strictly limited by coding style guide.
Potential risk	ASM syntax is compiler and compiler version dependent.
Prevention of risk	Component release test and delivery test shows correct behavior.
Note	-

Table 2-8 CW_008

2.9 Empty Source File

Deviation ID	CW_009
Example compiler warning strings	“empty source file“
Reason	According to AUTOSAR the BSW can be configured for various use cases (e.g. precompile, postbuild). The configuration data of these use cases is generated into separate files. Depending on the use case these files might be empty.
Potential risk	The build environment has to deal with empty translation units which might be considered as invalid syntax. This could result in compiler errors.
Prevention of risk	Component release test and delivery test shows correct behavior.
Note	Workaround: The user can explicitly remove the empty source file from his build steps.

Table 2-9 CW_009

2.10 Conversion to type ‘abc’ from int may alter its value

Deviation ID	CW_010
Example compiler warning strings	“conversion to ‘uint16 {aka short unsigned int} from ‘int’ may alter its value [-Wconversion]”
Reason	MICROSAR makes heavy use of integer types smaller than `int`; arithmetic operations cause integer promotion, resulting in implicit conversions back to the smaller type. This causes such warnings. Even `x += 2U`; causes a warning (if `x` is “smaller” than `int`, e.g. `unsigned short int`). However, this is allowed by MISRA’s Essential type model and by Rule 10.4.
Potential risk	A truncation occurs with unsigned values, unspecified behavior on signed values. However, this risk is inherent to arithmetic operations in general.
Prevention of risk	MISRA analysis; adhering to MISRA Rules related to “essential type model”. For signed values, additional overflow checks are performed with QAC at component level (e.g. Message IDs 2800…2803)
Note	-

Table 2-10 CW_010

2.11 Missing braces around initializer

Deviation ID	CW_011
Example compiler warning strings	“missing braces around initializer”
Reason	MISRA C:2012 Rule 9.2 explicitly states that a “zero” initializer `{0}` is compliant. In this specific use case (zero-initialization) it is convenient, expressive, and more concise to use such a shortened notation regardless of an aggregate’s actual structure. This deviation only applies to that specific initializer.
Potential risk	In this specific use case, there is no risk, because values get zero-initialized as intended, no matter how many braces or how many zeroes would actually be necessary.
Prevention of risk	N/A
Note	-

Table 2-11 CW_011

3 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com